Compiler Construction CCP

CCP Report

Section: A

Submitted By:

Syed Shayan Arshad	2023-CS-625
Adeel Ul Rehman	2023-CS-626
Hammad Khalid	2023-CS-637

Submitted To:
Mr. Aizaz Akmal

Dated: 10th June 2026

Department of Computer Science
University of Engineering and Technology Lahore, New Campus

1. Introduction

Syntax analysis is a fundamental phase of compiler design that checks whether a given input string follows the rules defined by a Context-Free Grammar (CFG). Different parsing methods have different capabilities in handling recursion, ambiguity, and parsing conflicts. This report examines five grammars using LL(1), LR(0), SLR(1), LR(1), and LALR(1) parsing techniques, highlighting their limitations, conflicts, and suitability for each grammar.

2. Grammar Properties Overview

G1
B → B or T | T
T → T and F | F
F → true | false | id
Properties: Left recursion in B and T. Unambiguous. No common prefixes.
Left Recursion Eliminated:
B → T B'   B' → or T B' | ε   T → F T'   T' → and F T' | ε   F → true | false | id

G2
S → i E t S | i E t S e S | a
E → b
Properties: No left recursion. Ambiguous, classic dangling-else. Common prefix 'i E t S' causes FIRST/FIRST conflict and shift-reduce conflict. Left-factoring cannot resolve the inherent ambiguity.

G3
L → L , id | id
Properties: Left recursion. Unambiguous. No common prefixes.
Transformed: L → id L'   L' → , id L' | ε

G4
S → id (Args)
Args → Args , id | id | ε
Properties: Left recursion in Args. Unambiguous. ε-production in Args.
Transformed: Args → id Args' | ε   Args' → , id Args' | ε

G5
S → StmtList   StmtList → Stmt StmtList | ε   Stmt → id = E ;
E → E + T | T   T → id | num
Properties: Left recursion in E. Unambiguous.
Transformed: E → T E'   E' → + T E' | ε

3. Parsing Technique Analysis

3.1 LL(1)

- **G1:** Can be parsed successfully after removing left recursion. The predictive parsing table contains no conflicting entries.
- **G2:** Cannot be parsed using LL(1). Even after left factoring, the dangling-else structure introduces a FIRST/FOLLOW conflict.
- **G3:** Becomes LL(1) after eliminating left recursion.
- **G4:** Suitable for LL(1) once left recursion is removed from Args.
- **G5:** Works with LL(1) after transforming the expression grammar to remove left recursion.

3.2 LR(0)

- G1: Generates a shift-reduce conflict because the parser cannot decide between reducing a completed production or shifting the operator or.
- G2: Produces a shift-reduce conflict due to the ambiguous placement of else.
- G3: Not LR(0) because reduction and shifting on comma may occur in the same state.
- G4: The ε-production causes uncertainty between reducing and continuing the parse.
- G5: Expression productions involving + create shift-reduce conflicts.

3.3 SLR(1)

- G1: FOLLOW sets are sufficient to eliminate LR(0) conflicts.
- G2: Ambiguity remains unresolved; shift-reduce conflict still exists.
- G3: Successfully parsed without conflicts.
- G4: Handles recursive argument lists correctly.
- G5: FOLLOW-based reductions remove LR(0) conflicts.

3.4 LR(1)

- G1: Accepted without parsing conflicts because LR(1) uses precise lookahead symbols.
- G2: Remains problematic since ambiguity is inherent in the grammar rather than caused by insufficient lookahead.
- G3: Successfully parsed; LR(1) lookahead resolves comma-based ambiguity.
- G4: Handles ε-productions without conflict.
- G5: Expression grammar accepted without conflicts.

3.5 LALR(1)

- G1: Parsed successfully with a compact parsing table.
- G2: Still fails because ambiguity cannot be removed by merging LR(1) states.
- G3: No conflicts occur after state merging.
- G4: Supports ε-productions and recursive lists efficiently.
- G5: Expression grammar is handled correctly.

4. Applicability Summary Table

Grammar	LL(1)	LR(0)	SLR(1)	LR(1)	LALR(1)
G1	✓ (after LR elim.)	✗ S/R conflict	✓	✓	✓
G2	✗ Ambiguous	✗ S/R conflict	✗ S/R conflict	✗ Ambiguous	✗ Ambiguous
G3	✓ (after LR elim.)	✗ S/R conflict	✓	✓	✓
G4	✓ (after LR elim.+LF)	✗ S/R+ε conflict	✓	✓	✓
G5	✓ (after LR elim.)	✗ S/R conflict	✓	✓	✓

LR elim. = Left Recursion Elimination   LF = Left Factoring   S/R = Shift-Reduce

5. Conflict Identification and Classification

5.1 Conflict Types

Shift-Reduce (S/R): State has both a shift item and a reduce item for the same lookahead.
Reduce-Reduce (R/R): Two complete items compete on the same lookahead.
FIRST/FIRST (LL only): Two alternatives for the same non-terminal have overlapping FIRST sets.
FIRST/FOLLOW (LL only): A → ε alternative exists and FIRST of another alternative overlaps FOLLOW(A).

5.2 Conflict Summary

Grammar	Technique	Conflict
G1	LR(0)	S/R
G2	All	Inherent Ambiguity
G2	LL(1)	FIRST/FOLLOW
G2	LR/SLR/LALR	S/R
G3	LR(0)	S/R
G4	LR(0)	S/R + ε
G5	LR(0)	S/R

6. Comparative Evaluation

6.1 Capability Comparison

Capability	LL(1)	LR(0)	SLR(1)	LR(1)	LALR(1)
Handles left recursion	No	Yes	Yes	Yes	Yes
Resolves S/R conflict	N/A	No	Often	Best	Usually
Resolves R/R conflict	N/A	No	No	Yes	Sometimes
Handles ambiguity	No	No	No	No	No
Lookahead used	1 token	None	FOLLOW sets	Exact sets	Merged sets

7. Final Justification

Grammar	Challenge	Recommended
G1	Left recursion	LALR(1)
G2	Ambiguity	Rewrite + LALR(1)
G3	Left recursion	LALR(1) / SLR(1)
G4	LR + ε-production	LALR(1)
G5	Left recursion in E	LALR(1)

8. Sample Derivations and Parsing Traces

8.1 G4 — SLR(1) Parse Trace for 'id(id,id)'

#	Stack	Input	Action
1	0	id (id , id) $	Shift id
2	0 id	(id , id) $	Shift (
3	0 id (	id , id) $	Shift id
4	0 id (id	, id) $	Reduce Args → id
5	0 id (Args	, id) $	Shift ,
6	0 id (Args ,	id) $	Shift id

7	0 id (Args , id	) $	Reduce Args → Args , id
8	0 id (Args	) $	Shift)
9	0 id (Args)	$	Reduce S → id(Args)
10	0 S	$	ACCEPT

9. Conclusions

- LR(0) parsing is not suitable for the given grammars because shift-reduce conflicts appear frequently.
- LL(1) can successfully parse G1, G3, G4, and G5 after applying transformations such as left recursion elimination and left factoring.
- Grammar G2 remains problematic due to the dangling-else ambiguity and cannot be parsed correctly without rewriting the grammar.
- SLR(1) provides better conflict handling than LR(0) and works for all grammars except G2.
- LR(1) offers the highest parsing power among the studied techniques and correctly handles all non-ambiguous grammars.
- LALR(1) achieves nearly the same capability as LR(1) while requiring fewer parser states.
- G1, G3, G4, and G5 are most effectively handled by LALR(1) because it balances efficiency and parsing strength.

Date: CC CCP Day:

All Grammers

Grammer 1:-

$B \rightarrow B \text{ or } T \mid T$

$T \rightarrow T \text{ and } F \mid F$

$F \rightarrow \text{true} \mid \text{false} \mid \text{id}$

Augmented Grammer:-

1. $B' \rightarrow B$
2. $B \rightarrow B \text{ or } T$
3. $B \rightarrow T$
4. $T \rightarrow T \text{ and } F$
5. $F \rightarrow \text{true}$
6. $F \rightarrow \text{false}$
7. $F \rightarrow \text{id}$
8. $T \rightarrow F$

Left Recursion:-

As there is left recursion in $B \rightarrow B \text{ or } T \mid T$ and $T \rightarrow T \text{ and } F \mid F$, so after removing left recursion we get:-

$B' \rightarrow B$ $\quad$ fi(B') = {true, false, id}

$B \rightarrow TB''$ $\quad$ fi(B) = {true, false, id}

$B'' \rightarrow \text{or } TB \mid \varepsilon$ $\quad$ fi(B'') = {or, ε}

$T \rightarrow FT'$ $\quad$ fi(T) = {true, false, id}

$T' \rightarrow \text{and } FT' \mid \varepsilon$ $\quad$ fi(T') = {and, ε}

$F \rightarrow \text{true} \mid \text{false} \mid \text{id}$ $\quad$ fi(F) = {true, false, id}

fo(B') = {\$}

fo(B) = {\$, or}

fo(T) = {\$, or, and}

fo(F) = {\$, or, and}

Ambiguity:-

There is no ambiguity in this grammer. and also no common prefix.

Date: Day:

Grammer 2 :-

S → iEtS | iEtSeS | a
E → b

Augmented Grammer :-

1. S' → S
2. S → iEtS
3. S → iEtSeS
4. S → a
5. E → b

Left Recursion :-

There is no left recurssion in this grammer

Ambiguity :-

This grammer has classic dangling-else ambiguity and also have **common prefix**. So after removing ambiguity with matched/unmatched and common prefix with **left factored** we get;

S → M | U
M → iEtMeM | a
U → iETU'
U' → S | MeU
E → b

fi(S) = {i, a}
fi(M) = {i, a}
fi(U) = {i}
fi(U') = {i, a}
fi(E) = {b}
S' → S   fi(S') = {i, a}

fo(S') = {$}
fo(S) = {e, $}
fo(E) = {t}

Date: Day:

Grammer 3 :-

L → L,id | id

Augmented Grammer :-

1. L' → L
2. L → L,id
3. L → id

Left Recursion :-

As there is left recursion in L → L,id, so after removing it we get:

L' → L
L → id L''
L'' → ,id L'' | ε

Ambiguity and Common prefixes :-

There is **no** ambiguity and common prefixes in this grammer.

fi(L') = {id}
fi(L) = {id}
fi(L'') = {, | ε}
fo(L') = {$}
fo(L) = {$ | ,}

CamScanner

Date: Day:

Grammer 4:-

S → id(Args)

Args → Args, id | id | ε

Augmented Grammer:-

① S' → S

② S → id(Args)

③ Args → Args, id

④ Args → id

⑤ Args → ε

Left Recursion:-

As there is left recursion in Args → Args, id | id so after removing it we get;

S' → S fi(S') = {id}

S → id(Args) fi(S) = {id}

Args → id Args' | ε fi(Args) = {id, ε}

Args' → , id Args' | ε fi(Args') = { , | ε}

Ambiguity and Common prefix:-

There is no ambiguity and common prefix in this grammer.

fo(S') = {$}

fo(S) = {$}

fo(Args) = { , |) }

Date: Day:

Grammer 5:-

S → StmtList

StmtList → Stmt StmtList | ε

Stmt → id = E;

E → E + T | T

T → id | num

Augmented Grammer:-

① S' → S fo(S') = {$}

② S → StmtList fo(S) = {$}

③ StmtList → Stmt StmtList

④ StmtList → ε fo(StmtList) = {$}

⑤ Stmt → id = E ; fo(Stmt) = {id, $}

⑥ E → E + T fo(E) = {+, ;}

⑦ E → T

⑧ T → id fo(T) = {+, ;}

⑨ T → num

~~Ambiguity and~~ Recursion:-

As this grammer has recursion E → E + T | T, so after removing it we get:-

S' → S fi(S) = {id, ε}

S → Stmt List fi(S) = {id, ε}

StmtList → Stmt StmtList | ε fi = {id, ε}

Stmt → id = E; fi(Stmt) = {id}

E → TE' fi(E) = {id, num}

E' → +TE' | ε fi(E') = {+, ε}

T → id | num fi(T) = {id, num}

This grammer has no ambiguity and common prefixes

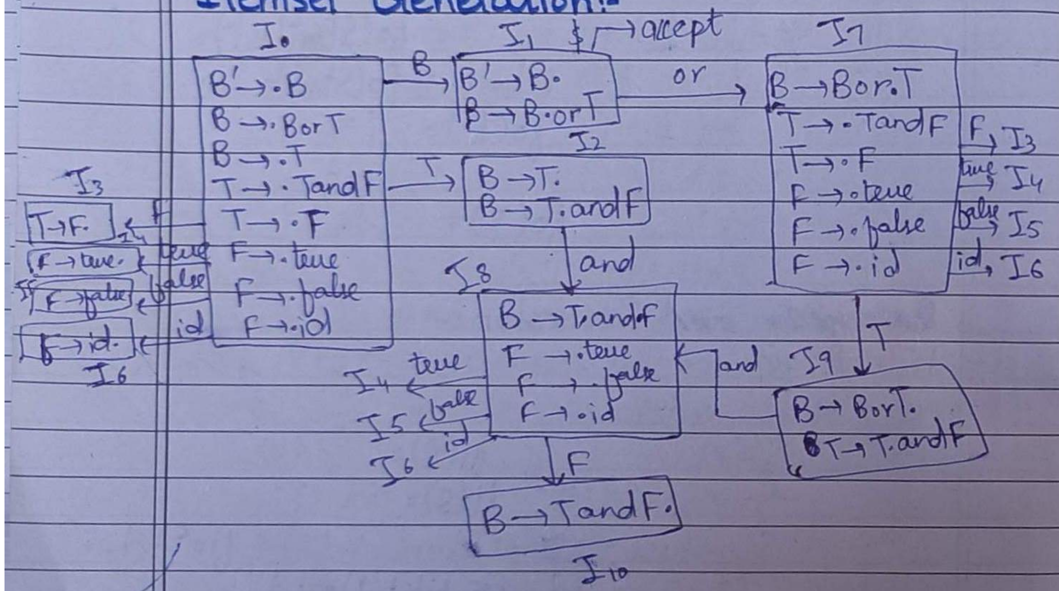
Date: Day:

LR(0) and SLR(1)

1) Grammer 1:

① $B' \to B$
② $B \to B \text{ or } T$
③ $B \to T$
④ $T \to T \text{ and } F$ ⑧ $T \to F$
⑤ $F \to$ true ⑥ | false | id ⑦

Itemset Generation:-



Date: Day:

Action Go-To Table:-

LR(0):-

States	or	and	true	false	id	$	B	T	F
0			S4	S5	S6		1	2	3
1	S7					accept			
2	r3	S8	r3	r3	r3	r3			
3	r8	r8	r8	r8	r8	r8			
4	r5	r5	r5	r5	r5	r5			
5	r6	r6	r6	r6	r6	r6			
6	r7	r7	r7	r7	r7	r7			
7			S4	S5	S6			9	3
8			S4	S5	S6				10
9	r2	S8	r2	r2	r2	r2			
10	r4	r4	r4	r4	r4	r4			

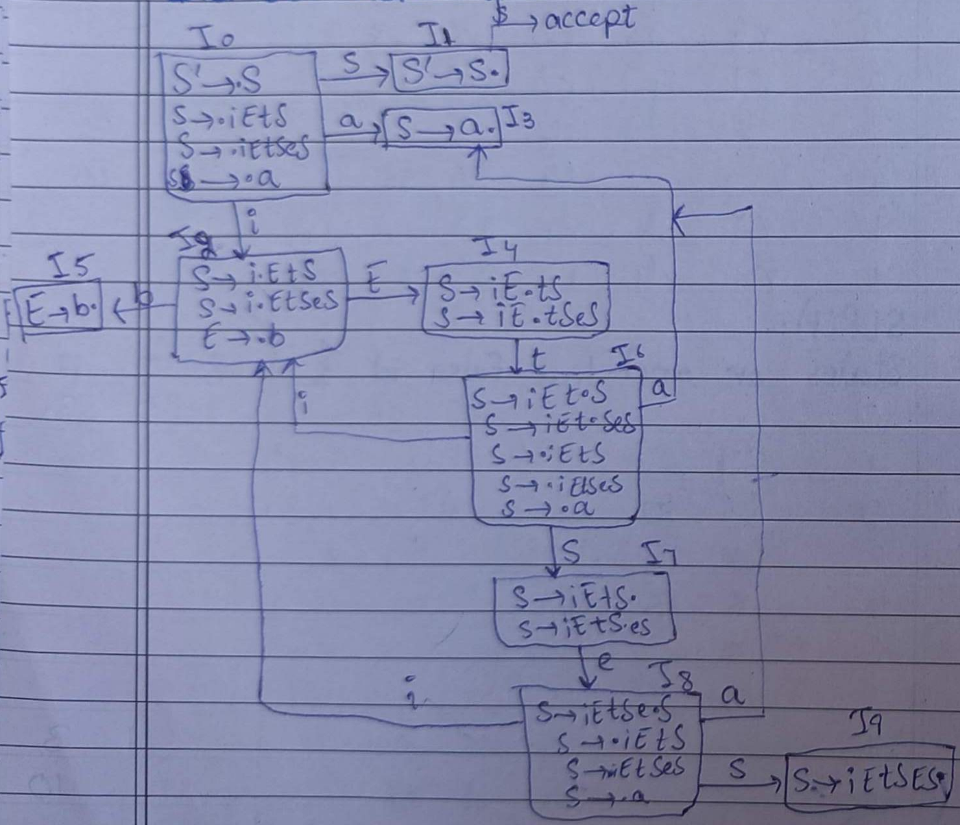
SLR(1):-

States	or	and	true	false	id	$	B	T	F
0			S4	S5	S6		1	2	3
1	S7					accept			
2	r3	S8				r3			
3	r8	r8				r8			
4	r5	r5				r5			
5	r6	r6				r6			
6	r7	r7				r7			
7			S4	S5	S6			9	3
8			S4	S5	S6				10
9	r2	S8				r2			
10	r4	r4				r4			

2) Grammer 2:

(1) $S' \to S$
(2) $S \to iEtS$
(3) $S \to iEtSeS$
(4) $S \to a$
(5) $E \cancel{S} \to b$

Itemset Generation:-



Action Go-To Table:-

LR(0):-

States	i	t	a	b	e	$	S	E
0	S2		S3				1	
1						accept		
2				S5				4
3	r4	r4	r4	r4	r4	r4		
4		S6						
5	r5	r5	r5	r5	r5	r5		
6	S2		S3				7	
7	r2	r2	r2	r2	S8	r2		
8	S2		S3				9	
9	r3	r3	r3	r3	r3	r3		

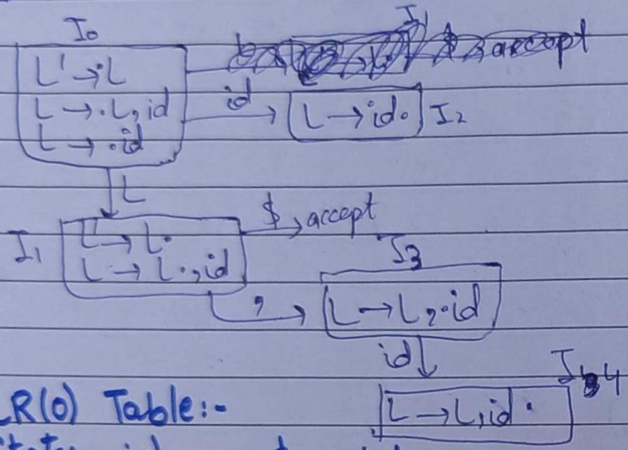
SLR(1)

States	i	t	a	b	e	$	S	E
0	S2		S3				1	
1						accept		
2				S5				4
3					r4	r4		
4		S6						
5		r5						
6	S2		S3				7	
7					S8/r2	r2		
8	S3		S3				9	
9					r3	r3		

3) Grammer 3:-

① $L' \rightarrow L$
② $L \rightarrow L,id$
③ $L \rightarrow id$

Itemset Generation:-



LR(0) Table:-

States	id	,	$	L
0	S2			1
1		S3	accept	
2	r3	r3	r3	
3	S4			
4	r2	r2	r2	

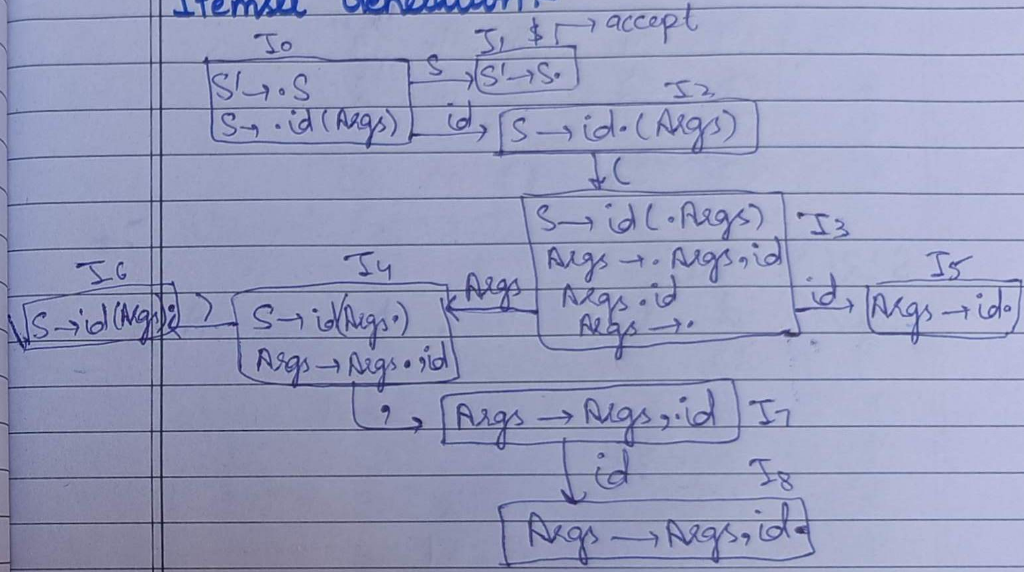
SLR(1) Table:-

States	id	,	$	L
0	S2			1
1		S3	accept	
2		r3	r3	
3	S4			
4		r2	r2	

4) Grammer 4:-

① $S' \rightarrow S$
② $S \rightarrow id(Args)$
③ $Args \rightarrow Args,id$
④ $Args \rightarrow id$
⑤ $Args \rightarrow \varepsilon$

Itemset Generation:-



LR(0) Table:-

States	id	(	)	,	$	S	Args
0	S2					1	
1					accept		
2		S3					
3	r5	r5	r5	r5	r5		4
4			S6	S7			
5	r4	r4	r4	r4	r4		
6	r2	r2	r2	r2	r2		
7	S8						
8	r3	r3	r3	r3	r3		

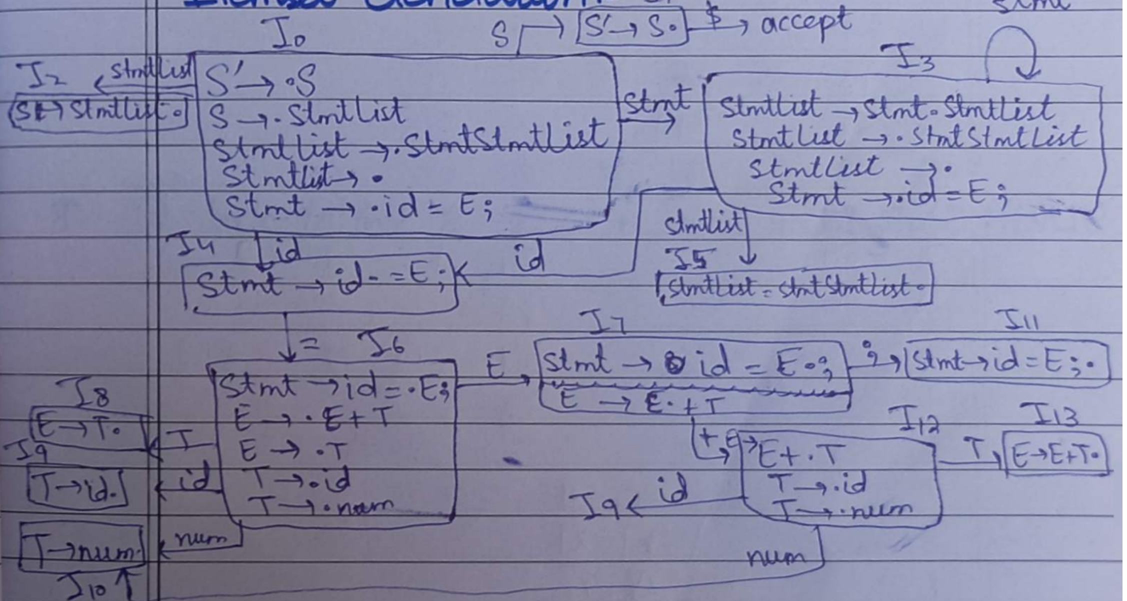
SLR(1) Table

States	id	(	)	,	$	S	Args
0	S2					1	
1					accept		
2		S3					
3	S5		r5	r5			4
4			S6	S7			
5			r4	r4			
6					r2		
7	S8						
8			r3	r3			

5) Grammar 5:-

1. $S' \to S$
2. $S \to StmtList$
3. $StmtList \to Stmt\ StmtList$
4. $StmtList \to \epsilon$
5. $Stmt \to id = E;$
6. $E \to E + T$
7. $E \to T$
8. $T \to id$
9. $T \to num$

Itemset Generation:-



LR(0) Action Go-To Table.

State	id	=	num	+	;	$	S	StmtList	Stmt	E	T
0	S4	r4	r4	r4	r4	r4	1	2	3		
1						accept					
2	r2	r2	r2	r2	r2	r2					
3	S4	~~r2~~ r4	~~r2~~ r4	~~r2~~ r4	~~r2~~ r4	~~r2~~ r4		5	3		
4		S6									
5	r3	r3	r3	r3	r3	r3					
6	S9		S10							7	8
7				S12	S11						
8	r7	r7	r7	r7	r7	r7					
9	r8	r8	r8	r8	r8	r8					
10	r9	r9	r9	r9	r9	r9					
11	r5	r5	r5	r5	r5	r5					
12	S9		S10								13
13	r6		r6	r6	r6	r6					

SLR(1):

State	id	=	num	+	;	$	S	Stmtlist	Stmt	E	T
0	S4					r4	1	2	3		
1						accept					
2						r2					
3	S4					r4		5	3		
4		S6									
5						r3					
6	S9		S10							7	8
7				S12	S11						
8				r7	r7						
9				r8	r8						
10				r8	r9						
11	r5					r5					
12	S9		S10								13
13				r6	r6						

LL(1)

Grammer 1:

B → B or T | T

T → T and F | F

F → true | false | id

As After Removing Left Recursion and left factoring, we get:

B' → B

B → TB''

B'' → or TB | ε

T → FT'

T' → and FT' | ε

F → true | false | id

Now using First and Follow of G1:

Parse Table:

	true	false	id	or	and	$
B	B→TB'	B→TB'	B→TB'			
B'				B'→orTB		B'→ε
T	T→FT'	T→FT'	T→FT'			
T'				T'→ε	T'→andFT'	T'→ε
F	F→true	F→false	F→id			

G1 gramer is LL1 because its parsing table contains no Conflict

G2 is

$S \rightarrow iEtS \mid iEtSeS \mid a$
$E \rightarrow b$

Augmented Gramer:

$S' \rightarrow S$
$S \rightarrow iEtS$
$S \rightarrow iEtSeS$
$S \rightarrow a$
$E \rightarrow b$

As the given gramar Contains left recursion, so after removing this

$S' \rightarrow S$
$S \rightarrow iEtSS'' \mid a$
$S'' \rightarrow eS \mid \varepsilon$
$E \rightarrow b$

first(S') = $\{i, a\}$
first(S) = $\{i, a\}$
first(S'') = $\{e, \varepsilon\}$
first(E) = $\{b\}$

Follow(S') = $\{\$\}$
Follow(S) = $\{e, \$\}$

Follow(S") : {e, $}
Follow(E) = {t}

Passing Table.

	i	a	b	e	t	$
S	S→iEtSS"	S→a				
S'				S'→eS/ε		
E			E→b			

As there is a Conflict in S'→eS/ε so the gramer is not LL1.

G3:

L→L,id | id

Augmented :

L'→ L

L → L,id

L → id

After Removing Left factoring we get :

L' →.L

L → idL"

L" →,idL" | ε

First (L') = {id}
First(L) = {id}
First (L'') = { , | ε}
Follow (L') = {$}
Follow (L) = {$ | ,}

Parsing Table.

	id	,	$
L'	L' → L		
L	L → id L''		
L''		L'' → ,id L''	L'' → ε

G4:

S → id
Args → Args, id | id | ε

As After Removing left recursion we get:

S' → S
S → id (Args)
Args → id Args' | ε
Args → , idArgs | ε

Using First and Follow set:

Parsing Table:

	id	(	)	,	$
S'	S' → S				
S	S → id(Args)				
Args	Args → id Args'		Args → ε		Args → ε
Args'			Args' → ε	Args' → , id Args'	Args' → ε

Conclusion:

There is No Conflict in Table so gramar is LL(1).

G5:

$S \rightarrow StmtList$

$StmtList \rightarrow Stmt\ StmtList \mid \varepsilon$

$Stmt \rightarrow id = E\ ;$

$E \rightarrow E + T \mid T$

$T \rightarrow id \mid num$

After performing Augmented Gram and left Recussion we get:

$S' \rightarrow S$

$S \rightarrow StmtList$

$StmtList \rightarrow Stmt\ StmtList \mid \varepsilon$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow id \mid num$

Parsing Table:

	id	num	=	+	;	$
S'	$S' \to S$					$S' \to \varepsilon$
S	$S \to StmtList$					$S \to \varepsilon$
StmtList	$S \to Stmt\,StmtList$				$StmtList \to \varepsilon$	$StmtList \to \varepsilon$
Stmt	$Stmt \to id = E$					
E	$E \to TE'$	$E \to TE'$				
E'						
T	$T \to id$	$T \to num$		$E' \to TE'$	$E \to \varepsilon$	$E' \to \varepsilon$

The Gramer is LL1.

LR(1)

G1

$B \to B \text{ or } T \mid T$
$T \to T \text{ and } F \mid F$
$F \to \text{true} \mid \text{false} \mid \text{id}$

CFG to Augmeted Grammer Conversion

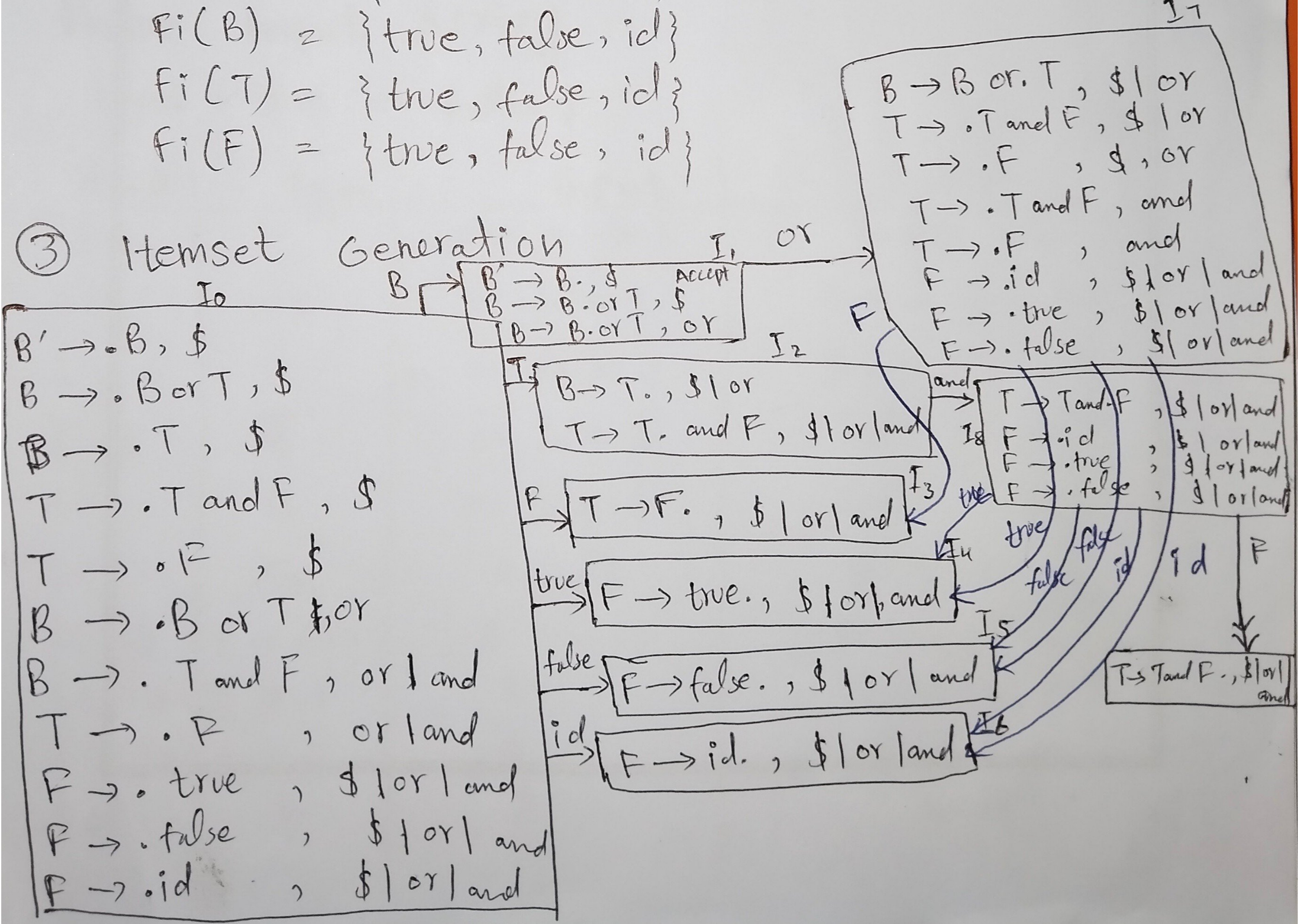
① $B' \to B$
$B \to B \text{ or } T \mid T$
$T \to T \text{ and } F \mid F$
$F \to \text{true} \mid \text{false} \mid \text{id}$

① $B' \to B$
② $B \to B \text{ or } T$
③ $B \to T$
④ $T \to T \text{ and } F$
⑤ $T \to F$
⑥ $F \to \text{true}$
⑦ $F \to \text{false}$
⑧ $F \to \text{id}$

② First set computation

$Fi(B') = \{\text{true}, \text{false}, \text{id}\}$
$Fi(B) = \{\text{true}, \text{false}, \text{id}\}$
$Fi(T) = \{\text{true}, \text{false}, \text{id}\}$
$Fi(F) = \{\text{true}, \text{false}, \text{id}\}$

③ Itemset Generation



Action goto Table

States	id	true	false	and	or	$	B'	B	T	F
I_0	S_6	S_4	S_5					1	2	3
I_1					S_7	Accept				
I_2				S_8	R_3	R_3				
I_3				R_5	R_5	R_5				
I_4				R_6	R_6	R_6				
I_5				R_7	R_7	R_7				
I_6				R_8	R_8	R_8				
I_7	S_6	S_4	S_5						9	3
I_8	S_6	S_4	S_5							10
I_9				S_8	R_2	R_2				
I_{10}				R_4	R_4	R_6				

Parse Input string

Input : true or false $

Stack	Sym	Input	Action
$0		True or false $	shift S_4
$04	true	or false $	Reduce F → true
$03	F	or false $	Reduce T → F
$02	~~T~~ B	or false $	Reduce B → T
$01	B	or false $	shift S_7
$017	B or	false $	Shift S_5
$0175	B or false	$	Reduce F → false
$0173	B or F	$	Reduce T → F
$0179	B or T	$	B → B or T
$01	B	$	Accept

$S \to iEtS \mid EtSeS \mid a$
$E \to b$

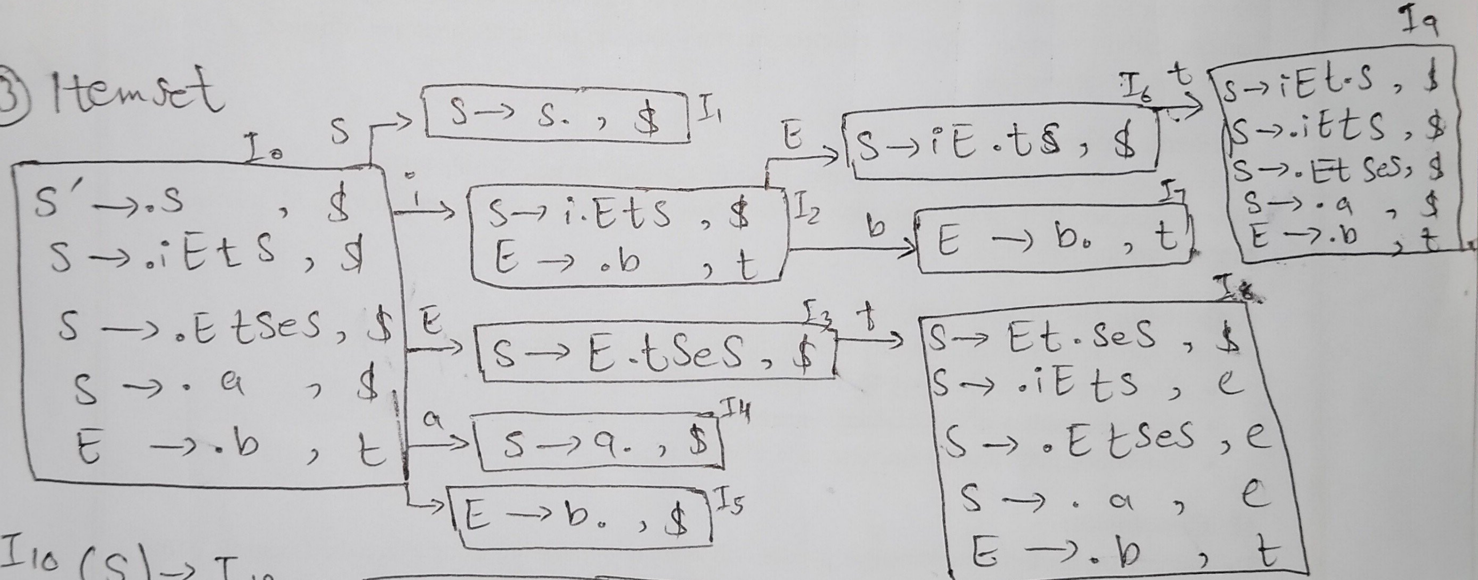
① Augmented Grammer Conversion

$S' \to S$
$S \to iEtS \mid iEtSeS \mid a$
$E \to b$

② First Set Computation

$Fi(S') = \{i, a\}$
$Fi(S) = \{i, a\}$
$Fi(E) = \{b\}$

③ Itemset



$I_{10}(S) \to I_{12} \to$ $\boxed{S \to iEtSeS. , \$}$
$I_{10}(i) \to I_2$
$I_{10}(a) \to I_3$

I_{11} $S \to I_{11}(t) \to I_{13} \to$

$S \to iEt.S$	$, \$ \mid e$
$S \to iEt.SeS$	$, \$ \mid e$
$S \to .EtSeS$	$, \$ \mid e$
$S \to .iEtS$	$, \$ \mid e$
$S \to .a$	$, \$ \mid e$

$I_{13}(S) \to I_{14} \to$

$S \to iEtS.$	$, \$ e$
$S \to iEtS.eS$	$, \$ \mid e$

$I_{13}(i) \rightarrow I_8$

$I_{13}(a) \rightarrow I_9$

$I_{14}(e) \rightarrow I_{15} \rightarrow \boxed{\begin{array}{ll} S \rightarrow iEtSe.S & , \$ \mid e \\ S \rightarrow .iEtS & , \$ \mid e \\ S \rightarrow .iEtSeS & \$ \mid e \\ S \rightarrow .a & \end{array}}$

$I_{15}(S) \rightarrow I_{16} \rightarrow \boxed{S \rightarrow iEtSeS. \quad \$ \mid e}$

$I_{15}(i) \rightarrow I_8$

$I_{15}(a) \rightarrow I_9$

Action Goto table

states	b	a	i	t	e	$	S	E	S′
I_0		S3	S2			Accept	1		
I_1									
I_2	S5							4	
I_3				R4					
I_4				R6					
I_5				R5					
I_6		S9	S8						
I_7					S10	R2	7		
I_8	S5							[illegible]	
I_9						R4			
I_{10}		S3	S2		R4		12		
I_{11}				S13	R2	R2			
I_{12}						R3			
I_{13}		S9	S8				14		
I_{14}					R15				
I_{15}							16		
I_{16}					R3	R3			

Input : ibta

State	symbol	input	Action
$0		ibta $	S2
$02	i	bta $	S5
$025	ib	ta $	Reduce t → b
$024	iE	ta $	S6
$0246	iEt	a $	S9
$02469	iEta	$	Reduce S → a
$02467	iEtS	$	Reduce S → SiEtS
	S	$	Accept
	S	$	

G3

$L \to L, id \mid id$

① Augmented Grammer

$L' \to L$

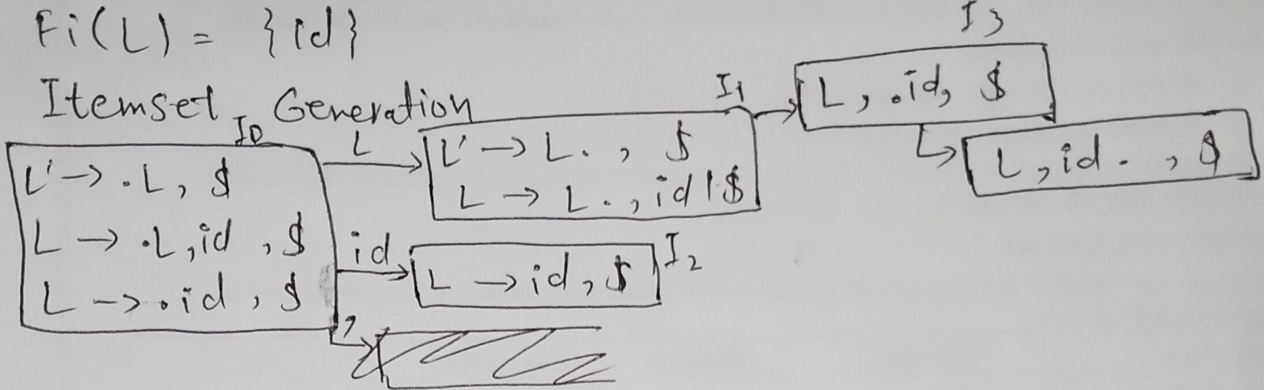
$L \to L, id \mid id$

② First Set Computation

$Fi(L') = \{id\}$

$Fi(L) = \{id\}$

③ Itemset Generation



Action Goto table

states	id	,	$	L	L'
I_0	S2			1	
I_1		S3			
I_2			R3		
I_3	S4				
I_4			R2		

Input Parsing

Input: id,id

stack	symbol	Input	Action
$0		id,id $	Shift S2
$02	id	,id $	Reduce R_3 $L \to id$
$01	L	, id $	Shift S3
$013	L,	id $	Shift S4
$0134	L, id	$	Reduce R_2 $L \to id$
$01	L	$	Accept

G4

S → id(Args)

Args → Args, id | id | ε

① Augmented Grammer

S′ → S

S → id(Args)

Args → Args, id | id | ε

① S′ → S

② S → id(Args)

③ Args → Args, id

④ Args → id

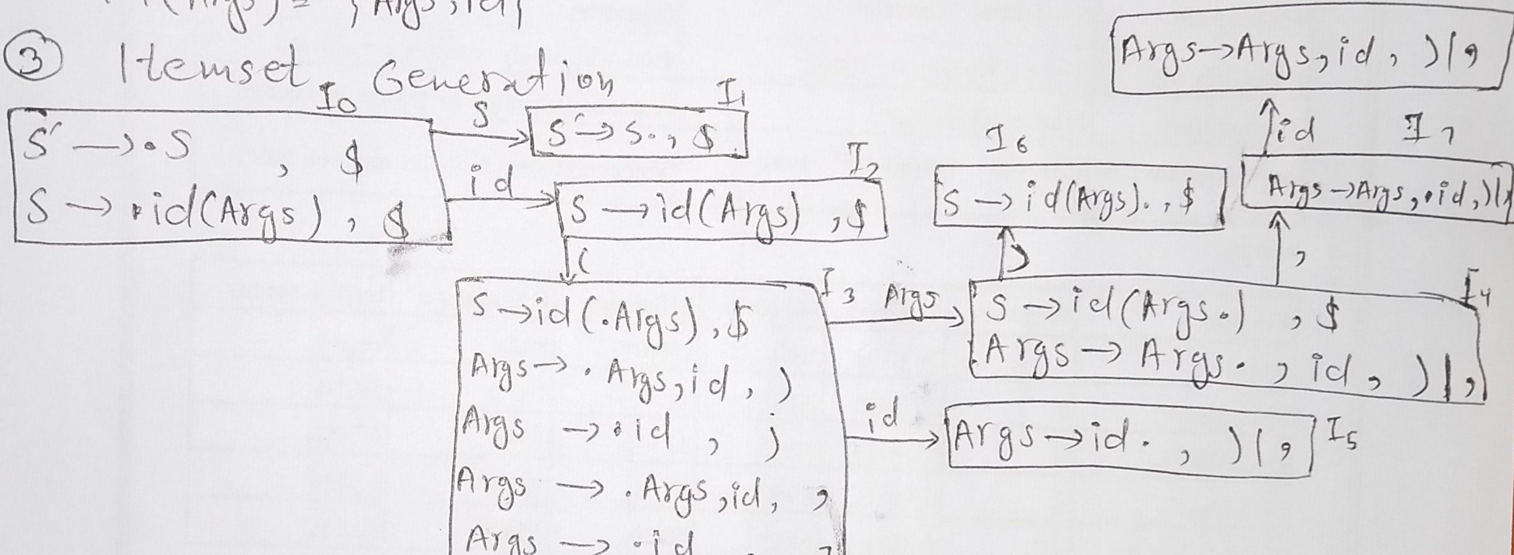
② First set Computation

Fi(S′) = {id}

Fi(S) = {id}

Fi(Args) = {Args, id}

③ Itemset Generation



Action Goto table

States	id	,	(	)	$	S′	S	Args
I_0	S2							
I_1					Accept		1	
I_2			S3					
I_3	S5							4
I_4		S7		S6				
I_5		R4		R4				
I_6					R2			
I_7	S8			R3				
I_8		R3						

Input Parsing

input : id(id)

stack	symbol	Input	Action
\$0		id(id)\$	s2
\$01	id	(id)\$	s3
\$023	id(	id)\$	s5
\$0235	id(id	)\$	Reduce id → Args
\$0234	id(Args	)\$	s6
\$02346	id(Args)	\$	reduce S → id(Args)
\$01	S	\$	Accept

G5

$S \rightarrow Stmlist$
$Stmlist \rightarrow stmt\ stmtlist \mid \varepsilon$
$stmt \rightarrow id = E ;$
$E \rightarrow E + T \mid T$
$T \rightarrow id \mid num$

① Augmented Grammer conversion

① $S' \rightarrow S$
② $S \rightarrow Stm List$
③ $stmList \rightarrow stmt$
④ $stmList \rightarrow \varepsilon$
⑤ $E \rightarrow E + T$
⑥ $E \rightarrow T$
⑦ $T \rightarrow id$
⑧ $T \rightarrow num$

② First set computation

$Fi(S') = \{id, \varepsilon\}$
$Fi(S) = \{id, \varepsilon\}$
$Fi(stmlist) = \{id, \varepsilon\}$
$Fi(E) = \{id, num\}$
$Fi(T) = \{id, num\}$

3) Itemset Generation

I_0

S' -> .S , $

S -> .stmlist , $

stmlist -> .stmt stmlist , $

stmt -> .id = E ; , id

stmlist -> ε

$I_0 \xrightarrow{S} I_1$: S' -> S. , $

$I_0 \xrightarrow{stmlist} I_2$: S -> stmlist. , $

$I_0 \xrightarrow{stmt} I_3$:

stmlist -> stmt .stmlist , $

stmlist -> .stmt stmtlist , $

stmtlist -> ε. , $

stmt -> .id = E ; , $ | ε | id

$I_0 \xrightarrow{id} I_4$: stmt -> id. = E ; , id

$I_0 \xrightarrow{ε}$ I_5: ~~stmlist -> ε , $~~

$I_3 \xrightarrow{stmlist} I_5$: stmlist -> stmt stmlist. , $

$I_3 \xrightarrow{stmt}$ stmlist -> ε. , $

$I_4 \xrightarrow{=} I_6$:

stmt -> id = .E ; , $ | id | ε

E -> .E + T , ; +

E -> .T ; +

E -> .T , +

$I_6 \xrightarrow{E} I_7$:

stmt -> id = E . ; , $ | id | ε

E -> E . + T , ; +

$I_6 \xrightarrow{T}$ E -> T. , + | ;

$I_6 \xrightarrow{id}$ T -> id. , ; +

I6(num) -> I10 -> T -> num. ; +

I7(;) -> I11 stmt -> id = E ; . $ | id | ε

I7(+) -> I12 ->

E -> E + .T , + ;

T -> .id , + ;

T -> .num , + ;

I12(T) -> I13 E -> E + T. , + ;

I12(id) -> I9

I12(num) -> I10

Action Goto table

States	=	+	;	,	id	num	$	E	T	StmList	Stmt	S	E
I_0										2	3	2	
I_1						A	Accept						
I_2							R2						
I_3													
I_4	S6									5	3		
I_5							R3						
I_6					S9			7	8				
I_7		S12	S11										
I_8		R6	R6										
I_9		R7	R7										
I_{10}		R8	R8										
I_{11}					R4		R4						
I_{12}					S9	S10							
I_{13}		R5	R5										
I_{14}							R2						
~~I~~													

input : id:id;

Stack	Symbol	Input	Action
$0		id:id $	S4
$04	id	:id $	S6
$046	id:	id $	S9
$0469	id:id	; $	Reduce T→id
$0468	id : T	; $	Reduce E → T
$ 0467	id : E	; $	S11
$04671	id: E;	$	Reduce Stmt→id : E;
$03	Stmt	ε $	S14
$0314	Stmt	$	Reduce stmlist→ stmtstmList
$02	stmlist	$	Reduce S→stmList
	S	$	Accept

G2 LALR(1) Merging States

$\{I_1, I_7\} \rightarrow L1$

$\{I_2, I_8\} \rightarrow L0$

$\{I_5, I_{10}\} \rightarrow L5$

$\{I_6, I_{12}\} \rightarrow L6$

$\{I_9, I_4\} \rightarrow L7$

$\{I_{11}, I_{15}\} \rightarrow L8$

$\{I_1, I_{10}\} \rightarrow L9$

Parsing Table

State	i	t	e	a	b	$	S	E
0	S1			S2			3	
1					S4			
2			r3			r3		5
3						Accept		
4		r4						
5								
6	S1	r6		S2			7	
7		r/S8				r1		
8	S1			S2			9	
9		r2				r2		

LALR(1) Conflicts state 7 e: r1/S8

All other Grammars G1, G3, G4 and G5 have same LALR because there is no same production and different lookahead in their LR(1)'s itemset.